

Epic 4: Model Building

Story 6: Saving the Best Model for Deployment

After evaluating all the machine learning models, XGBoost was selected as the final model because it achieved the highest prediction accuracy (96.55%) and demonstrated reliable performance on the flood prediction dataset. To avoid retraining the model every time a prediction is required, the trained model was saved using the Joblib library.

Saving the Trained Model

The trained XGBoost model is stored as `floods.save`, while the fitted `StandardScaler` is saved as `transform.save`. Saving both the model and the scaler ensures that the same preprocessing steps and prediction logic used during training are consistently applied during deployment.

Model Deployment Process

During real-time prediction, the Flask application performs the following steps:

1. Loads the saved `StandardScaler` (`transform.save`).
2. Standardizes the user-provided weather input using the saved scaler.
3. Loads the trained XGBoost model (`floods.save`).
4. Generates the flood prediction based on the processed input.
5. Displays the prediction result (Flood / No Flood) to the user through the web interface.

Benefits

- Eliminates the need to retrain the model for every prediction.
- Ensures consistent preprocessing using the saved `StandardScaler`.
- Reduces prediction time and improves application efficiency.
- Provides accurate and reliable real-time flood risk predictions.
- Simplifies deployment and integration with the Flask web application.

Outcome

- Selected XGBoost as the final deployment model.
- Saved the trained model using Joblib.
- Saved the fitted `StandardScaler` for consistent preprocessing.

- Successfully prepared the model for integration into the Flask-based AI Flood Prediction System.

Figure 17: Saving the trained XGBoost model and StandardScaler using Joblib for deployment.

```
In [41]: metrics.confusion_matrix(y_test,p4)

array([[25,  1],
       [ 0,  3]], dtype=int64)

In [40]: print(metrics.accuracy_score(y_test,p4))

0.9655172413793104

In [42]: print(metrics.precision_score(y_test,p4))

0.75

In [43]: print(metrics.recall_score(y_test,p4))

1.0
```